

# COMPTE RENDU

Administration Réseaux et Programmation Système - TP4 - Processus et  
Parallélisme  
3e année Cybersécurité - École Supérieure d'Informatique et du  
Numérique (ESIN)  
Collège d'Ingénierie & d'Architecture (CIA)

**Étudiant :** HATHOUTI Mohammed Taha  
**Filière :** Cybersécurité  
**Année :** 2025/2026  
**Enseignants :** M. Noufel & M. ElAmrani  
**Date :** 28 février 2026

## Table des matières

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Exercice 1 : Création de processus</b>                                  | <b>2</b>  |
| 1.1      | Partie a — Deux fils affichant 1 à 50 et 51 à 100 . . . . .                | 2         |
| 1.2      | Partie b — Affichage ordonné de 1 à 100 . . . . .                          | 4         |
| <b>2</b> | <b>Exercice 2 : Simultanéité vs. séquentialité</b>                         | <b>5</b>  |
| 2.1      | Partie a — Équivalent C de <code>who &amp; ps &amp; ls -l</code> . . . . . | 5         |
| 2.2      | Partie b — Équivalent C de <code>who ; ps ; ls -l</code> . . . . .         | 7         |
| <b>3</b> | <b>Exercice 3 : Synchronisation de processus</b>                           | <b>9</b>  |
| 3.1      | Partie a — <code>creerL nb</code> : création en largeur . . . . .          | 9         |
| 3.2      | Partie b — <code>creerP nb</code> : création en profondeur . . . . .       | 10        |
| 3.3      | Partie c — <code>creerA hauteur</code> : arbre binaire . . . . .           | 12        |
| <b>4</b> | <b>Exercice 4 : Gestion des entrées/sorties</b>                            | <b>14</b> |
| 4.1      | Principe . . . . .                                                         | 14        |
| 4.2      | Version 1 — <code>fopen / fgetc</code> (E/S bufférisées) . . . . .         | 14        |
| 4.3      | Version 2 — <code>open / read</code> (E/S non bufférisées) . . . . .       | 15        |
| 4.4      | Conclusion . . . . .                                                       | 17        |
| <b>5</b> | <b>Exercice 5 : La commande <code>execvp</code></b>                        | <b>17</b> |
| 5.1      | Principe . . . . .                                                         | 17        |
| 5.2      | Résultat d'exécution . . . . .                                             | 18        |
| 5.3      | Analyse . . . . .                                                          | 18        |
|          | <b>Conclusion</b>                                                          | <b>18</b> |

# 1 Exercice 1 : Création de processus

## 1.1 Partie a — Deux fils affichant 1 à 50 et 51 à 100

L'objectif est de créer deux processus fils à partir d'un processus père via l'appel système `fork()`. Le premier fils affiche les entiers de 1 à 50, et le second de 51 à 100. Les deux fils s'exécutent en parallèle, ce qui signifie que l'ordre d'affichage n'est pas garanti, le système d'exploitation décide quel processus obtient le CPU en premier.

La valeur de retour de `fork()` permet de distinguer les rôles :

- **Dans le père** : `fork()` retourne le PID du fils (valeur  $> 0$ );
- **Dans le fils** : `fork()` retourne 0;
- **En cas d'erreur** : `fork()` retourne -1.

Le père appelle `wait(NULL)` deux fois afin d'attendre la terminaison de chacun de ses deux fils et d'éviter la création de processus zombies.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t fils1, fils2;
8
9     fils1 = fork();
10
11     if (fils1 < 0) {
12         perror("Erreur fork 1");
13         exit(1);
14     }
15
16     if (fils1 == 0) {
17         for (int i = 1 ; i < 51 ; i++) {
18             if (i == 1) {
19                 printf("Fils 1 : [ %d,", i);
20             } else if (i == 50) {
21                 printf(" %d ]\n\n", i);
22             } else {
23                 printf(" %d,", i);
24             }
25         }
26         exit(0);
27     }
28
29     fils2 = fork();
30
31     if (fils2 < 0) {
32         perror("Erreur fork 2");
33         exit(1);
34     }
35 }
```

```

36     if (fils2 == 0) {
37         for (int i = 51 ; i <= 100 ; i++) {
38             if (i == 51) {
39                 printf("Fils 2 : [ %d,", i);
40             } else if (i == 100) {
41                 printf(" %d ]\n\n", i);
42             } else {
43                 printf(" %d,", i);
44             }
45         }
46         exit(0);
47     }
48
49     wait(NULL);
50     wait(NULL);
51
52     printf("Pere : les deux fils ont termine.\n");
53
54     return 0;
55 }

```

## Résultats d'exécution

Comme les deux fils s'exécutent en parallèle, l'ordre d'affichage varie d'une exécution à l'autre. Les trois cas observés sont illustrés ci-dessous.

### Cas 1 — Le père affiche son message avant les fils (sans wait) :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4$ ./ex1
Père : les deux fils ont terminé.
Fils 1 : [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50 ]
Fils 2 : [ 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100 ]

```

FIGURE 1 – Exécution sans wait : le père se termine avant les fils

### Cas 2 — Fils 2 s'affiche avant Fils 1 :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4$ ./ex1
Fils 2 : [ 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100 ]
Fils 1 : [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50 ]
Père : les deux fils ont terminé.

```

FIGURE 2 – Exécution avec wait : Fils 2 obtient le CPU avant Fils 1

### Cas 3 — Fils 1 s'affiche avant Fils 2 (ordre attendu) :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4$ ./ex1
Fils 1 : [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50 ]
Fils 2 : [ 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100 ]
Père : les deux fils ont terminé.

```

FIGURE 3 – Exécution avec wait : Fils 1 obtient le CPU avant Fils 2

## Analyse

Ces trois cas illustrent bien le caractère **non déterministe** de l'exécution parallèle : même programme, résultats différents selon l'ordonnancement du système. Le `wait(NULL)` garantit que le père attend bien ses fils avant de se terminer, mais n'impose aucun ordre entre les fils eux-mêmes.

### 1.2 Partie b — Affichage ordonné de 1 à 100

Pour garantir l'affichage dans l'ordre 1, 2, 3, ..., 100, il faut que le père attende la fin du Fils 1 **avant** de créer le Fils 2. La modification est simple : on déplace le premier `wait(NULL)` entre les deux `fork()`, au lieu de les placer tous les deux à la fin. Ainsi, le Fils 2 n'est créé que lorsque le Fils 1 a terminé son affichage.

Listing 1 – ex1.2.c — Affichage ordonné de 1 à 100

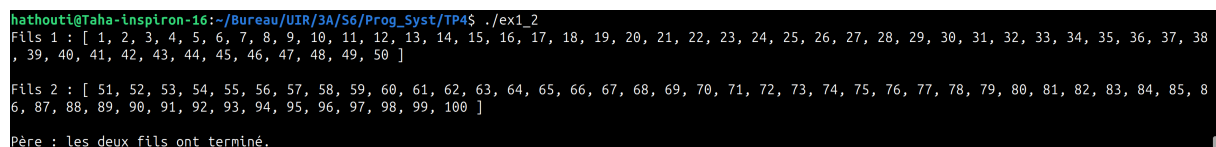
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t fils1, fils2;
8
9     fils1 = fork();
10
11     if (fils1 < 0) {
12         perror("Erreur fork 1");
13         exit(1);
14     }
15
16     if (fils1 == 0) {
17         for (int i = 1 ; i < 51 ; i++) {
18             if (i == 1) {
19                 printf("Fils 1 : [ %d,", i);
20             } else if (i == 50) {
21                 printf(" %d ]\n\n", i);
22             } else {
23                 printf(" %d,", i);
24             }
25         }
26         exit(0);
27     }
28
29     wait(NULL); /* Pere attend la fin de Fils 1 avant de creer
30                 Fils 2 */
31
32     fils2 = fork();
33
34     if (fils2 < 0) {
35         perror("Erreur fork 2");
36         exit(1);
37     }
```

```

36     }
37
38     if (fils2 == 0) {
39         for (int i = 51 ; i <= 100 ; i++) {
40             if (i == 51) {
41                 printf("Fils 2 : [ %d,", i);
42             } else if (i == 100) {
43                 printf(" %d ]\n\n", i);
44             } else {
45                 printf(" %d,", i);
46             }
47         }
48         exit(0);
49     }
50
51     wait(NULL);
52
53     printf("Pere : les deux fils ont termine.\n");
54
55     return 0;
56 }

```

## Résultat d'exécution



```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4$ ./ex1_2
Fils 1 : [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50 ]
Fils 2 : [ 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100 ]
Père : les deux fils ont terminé.

```

FIGURE 4 – Affichage garanti dans l'ordre : Fils 1 puis Fils 2

## Analyse

Le simple déplacement du `wait(NULL)` entre les deux `fork()` suffit à imposer l'ordre d'exécution. Le père bloque après la création du Fils 1 et n'avance qu'une fois celui-ci terminé. Fils 2 est donc créé séquentiellement après Fils 1, ce qui garantit un affichage de 1 à 100 sans interleaving. En contrepartie, les deux fils ne s'exécutent plus en parallèle.

## 2 Exercice 2 : Simultanéité vs. séquentialité

### 2.1 Partie a — Équivalent C de `who & ps & ls -l`

En shell, le `&` lance une commande en arrière-plan, ce qui signifie que les trois commandes s'exécutent en **parallèle**. Pour reproduire ce comportement en C, on crée trois processus fils avec `fork()`, chacun exécutant une commande via `execvp()`. Les trois `wait(NULL)` sont placés à la fin, après la création de tous les fils, afin de ne pas bloquer le père entre les créations.

`execvp()` remplace le processus fils courant par la commande système spécifiée. Il prend en paramètre le nom de la commande et un tableau d'arguments terminé par `NULL`.

Listing 2 – ex2\_1.c — Équivalent de `who & ps & ls -l`

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t f1, f2, f3;
8
9     f1 = fork();
10    if (f1 < 0) { perror("fork 1"); exit(1); }
11    if (f1 == 0) {
12        char *args[] = {"who", NULL};
13        execvp("who", args);
14        perror("execvp who"); exit(1);
15    }
16
17    f2 = fork();
18    if (f2 < 0) { perror("fork 2"); exit(1); }
19    if (f2 == 0) {
20        char *args[] = {"ps", NULL};
21        execvp("ps", args);
22        perror("execvp ps"); exit(1);
23    }
24
25    f3 = fork();
26    if (f3 < 0) { perror("fork 3"); exit(1); }
27    if (f3 == 0) {
28        char *args[] = {"ls", "-l", NULL};
29        execvp("ls", args);
30        perror("execvp ls"); exit(1);
31    }
32
33    wait(NULL);
34    wait(NULL);
35    wait(NULL);
36
37    return 0;
38 }

```

## Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex2$ ./ex2_1
hathouti seat0          2026-02-16 14:17 (login screen)
hathouti :1             2026-02-16 14:17 (:1)
hathouti pts/5          2026-02-24 11:56
total 32
-rw-rw-r-- 1 hathouti hathouti 824 févr. 28 00:06 ex1_2.c
-rwxrwxr-x 1 hathouti hathouti 18136 févr. 28 00:16 ex2_1
-rw-rw-r-- 1 hathouti hathouti 697 févr. 28 00:08 ex2_1.c
-rw-rw-r-- 1 hathouti hathouti 454 févr. 28 00:15 Makefile
  PID TTY          TIME CMD
 273894 pts/2    00:00:00 bash
 404557 pts/2    00:00:00 ex2_1
 404559 pts/2    00:00:00 ps
```

FIGURE 5 – Exécution parallèle : les sorties de `who`, `ps` et `ls -l` sont mélangées

## Analyse

On observe que les sorties des trois commandes sont entremêlées : `who` et `ls -l` s'affichent avant que `ps` ne termine. C'est le comportement attendu d'une exécution parallèle, identique au `&` du shell.

## 2.2 Partie b — Équivalent C de `who ; ps ; ls -l`

En shell, le `;` exécute les commandes **séquentiellement** : la suivante ne démarre que lorsque la précédente est terminée. Pour reproduire ce comportement, il suffit de placer un `wait(NULL)` après chaque `fork()`, forçant le père à attendre la fin de chaque fils avant de créer le suivant.

Listing 3 – `ex2.2.c` — Équivalent de `who ; ps ; ls -l`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t f1, f2, f3;
8
9     f1 = fork();
10    if (f1 < 0) { perror("fork 1"); exit(1); }
11    if (f1 == 0) {
12        char *args[] = {"who", NULL};
13        execvp("who", args);
14        perror("execvp who"); exit(1);
15    }
16    wait(NULL);
17
18    f2 = fork();
```



```

19     if (f2 < 0) { perror("fork 2"); exit(1); }
20     if (f2 == 0) {
21         char *args[] = {"ps", NULL};
22         execvp("ps", args);
23         perror("execvp ps"); exit(1);
24     }
25     wait(NULL);
26
27     f3 = fork();
28     if (f3 < 0) { perror("fork 3"); exit(1); }
29     if (f3 == 0) {
30         char *args[] = {"ls", "-l", NULL};
31         execvp("ls", args);
32         perror("execvp ls"); exit(1);
33     }
34     wait(NULL);
35
36     return 0;
37 }

```

## Résultat d'exécution

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex2$ ./ex2_2
hathouti seat0      2026-02-16 14:17 (login screen)
hathouti :1         2026-02-16 14:17 (:1)
hathouti pts/5      2026-02-24 11:56
  PID TTY          TIME CMD
 273894 pts/2    00:00:00 bash
 404964 pts/2    00:00:00 ex2_2
 404966 pts/2    00:00:00 ps
total 52
-rwxrwxr-x 1 hathouti hathouti 18136 févr. 28 00:16 ex2_1
-rw-rw-r-- 1 hathouti hathouti   697 févr. 28 00:08 ex2_1.c
-rwxrwxr-x 1 hathouti hathouti 18144 févr. 28 00:20 ex2_2
-rw-rw-r-- 1 hathouti hathouti   697 févr. 28 00:19 ex2_2.c
-rw-rw-r-- 1 hathouti hathouti   513 févr. 28 00:20 Makefile

```

FIGURE 6 – Exécution séquentielle : `who` puis `ps` puis `ls -l` dans l'ordre

## Analyse

Les sorties apparaissent dans l'ordre strict : d'abord `who`, ensuite `ps`, enfin `ls -l`. Aucun mélange n'est possible puisque chaque fils doit terminer avant que le suivant ne soit créé. Ce comportement est identique au ; du shell.

## 3 Exercice 3 : Synchronisation de processus

### 3.1 Partie a — creerL nb : création en largeur

Le programme `creerL` crée `nb` fils issus du **même père**. La structure obtenue est une arborescence à un seul niveau : un père avec `nb` fils directs. Le père effectue une boucle de `nb fork()`, et chaque fils affiche son PID et le PID de son père avant de terminer avec `exit(0)`. Les `wait(NULL)` sont regroupés à la fin pour que le père attende la terminaison de tous ses fils.

Listing 4 – `creerL.c` — Création de `nb` fils en largeur

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc != 2) {
8         fprintf(stderr, "Usage: %s nb\n", argv[0]);
9         exit(1);
10    }
11
12    int nb = atoi(argv[1]);
13    pid_t pid;
14
15    for (int i = 0 ; i < nb ; i++) {
16        pid = fork();
17        if (pid < 0) { perror("fork"); exit(1); }
18        if (pid == 0) {
19            printf("Fils %d : PID = %d, PID père = %d\n",
20                i + 1, getpid(), getppid());
21            exit(0);
22        }
23    }
24
25    for (int i = 0; i < nb; i++) {
26        wait(NULL);
27    }
28
29    printf("Père : PID = %d, tous les fils ont terminé.\n",
30        getpid());
31
32    return 0;
33 }
```

## Résultats d'exécution

Sans wait :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerL 6
Fils 1 : PID = 409268, PID père = 409267
Fils 2 : PID = 409269, PID père = 409267
Fils 3 : PID = 409270, PID père = 409267
Père : PID = 409267, le Processus Père à terminé.
Fils 5 : PID = 409272, PID père = 409267
Fils 4 : PID = 409271, PID père = 409267
Fils 6 : PID = 409273, PID père = 409267
```

FIGURE 7 – creerL 6 sans attente : le père se termine avant certains fils

Avec wait :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerL 6
Fils 1 : PID = 409347, PID père = 409346
Fils 2 : PID = 409348, PID père = 409346
Fils 3 : PID = 409349, PID père = 409346
Fils 4 : PID = 409350, PID père = 409346
Fils 5 : PID = 409351, PID père = 409346
Fils 6 : PID = 409352, PID père = 409346
Père : PID = 409346, tous les fils ont terminé.
```

FIGURE 8 – creerL 6 avec attente : le père attend tous ses fils

## Analyse

Sans `wait`, le père peut se terminer avant certains fils, comme observé dans la première capture où le message du père apparaît au milieu des affichages des fils. Avec `wait`, tous les fils terminent avant le père, garantissant une terminaison propre sans processus zombies.

## 3.2 Partie b — creerP nb : création en profondeur

Le programme `creerP` crée une chaîne de `nb` générations : le père crée un fils, qui crée lui-même un fils, et ainsi de suite. Chaque processus connaît le PID du processus initial grâce à la variable `pid_initial` héritée par `fork()`. Dans la branche père, on appelle `wait(NULL)` puis `exit(0)`, forçant chaque père à attendre son fils unique avant de se terminer.

Listing 5 – creerP.c — Création de nb générations en profondeur

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc != 2) {
8         fprintf(stderr, "Usage: %s nb\n", argv[0]);
```

```

9      exit(1);
10  }
11
12  int nb = atoi(argv[1]);
13  pid_t pid_initial = getpid();
14  pid_t pid;
15
16  printf("Processus initial : PID = %d\n", pid_initial);
17
18  for (int i = 0; i < nb; i++) {
19      pid = fork();
20      if (pid < 0) { perror("fork"); exit(1); }
21      if (pid == 0) {
22          printf("Génération %d : PID = %d, PID père = %d, PID
                initial = %d\n",
23                  i + 1, getpid(), getppid(), pid_initial);
24      } else {
25          wait(NULL);
26          exit(0);
27      }
28  }
29
30  exit(0);
31 }

```

## Résultats d'exécution

Sans wait :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerP 6
Processus initial : PID = 409840
Génération 1 : PID = 409841, PID père = 409840, PID initial = 409840
Génération 2 : PID = 409842, PID père = 409841, PID initial = 409840
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ Génération 3 : PID = 409843, PID père = 409842, PID initial = 409840
Génération 4 : PID = 409844, PID père = 409843, PID initial = 409840
Génération 5 : PID = 409845, PID père = 409844, PID initial = 409840
Génération 6 : PID = 409846, PID père = 409845, PID initial = 409840

```

FIGURE 9 – creerP 6 sans attente : le shell reprend la main avant la fin de la chaîne

Avec wait :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerP 6
Processus initial : PID = 409941
Génération 1 : PID = 409942, PID père = 409941, PID initial = 409941
Génération 2 : PID = 409943, PID père = 409942, PID initial = 409941
Génération 3 : PID = 409944, PID père = 409943, PID initial = 409941
Génération 4 : PID = 409945, PID père = 409944, PID initial = 409941
Génération 5 : PID = 409946, PID père = 409945, PID initial = 409941
Génération 6 : PID = 409947, PID père = 409946, PID initial = 409941

```

FIGURE 10 – creerP 6 avec attente : la chaîne s'affiche dans l'ordre

## Analyse

On remarque que le PID de chaque génération est celui du fils précédent + 1, confirmant la structure en chaîne. Le PID initial reste le même pour toutes les générations car il est transmis par héritage lors du `fork()`. Sans `wait`, le shell reprend la main avant que toutes les générations ne s'affichent.

### 3.3 Partie c — creerA hauteur : arbre binaire

Le programme `creerA` crée récursivement un arbre binaire de processus. Chaque nœud crée deux fils (gauche et droit) via `fork()`, et chacun appelle récursivement `creer_arbre` avec `hauteur - 1`. La récursion s'arrête quand `hauteur == 0`. Pour un arbre de hauteur  $h$ , le nombre total de processus créés est  $2^{h+1} - 1$ .

Listing 6 – `creerA.c` — Création d'un arbre binaire de processus

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 void creer_arbre(int hauteur_restante) {
7     if (hauteur_restante == 0) return;
8
9     pid_t fils1, fils2;
10
11     fils1 = fork();
12     if (fils1 < 0) { perror("fork fils1"); exit(1); }
13     if (fils1 == 0) {
14         printf("Fils gauche : PID = %d, PID père = %d\n",
15             getpid(), getppid());
16         creer_arbre(hauteur_restante - 1);
17         wait(NULL); wait(NULL);
18         exit(0);
19     }
20
21     fils2 = fork();
22     if (fils2 < 0) { perror("fork fils2"); exit(1); }
23     if (fils2 == 0) {
24         printf("Fils droit : PID = %d, PID père = %d\n",
25             getpid(), getppid());
26         creer_arbre(hauteur_restante - 1);
27         wait(NULL); wait(NULL);
28         exit(0);
29     }
30
31     wait(NULL);
32     wait(NULL);
33 }
34
35 int main(int argc, char *argv[]) {
36     if (argc != 2) {
```

```

37     fprintf(stderr, "Usage: %s hauteur\n", argv[0]);
38     exit(1);
39 }
40
41 int hauteur = atoi(argv[1]);
42 printf("Racine : PID = %d\n", getpid());
43 creer_arbre(hauteur);
44
45 return 0;
46 }

```

## Résultats d'exécution

Sans wait :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerA 3
Racine : PID = 411905
Fils gauche : PID = 411906, PID père = 411905
Fils droit : PID = 411907, PID père = 411905
Fils gauche : PID = 411908, PID père = 411906
Fils gauche : PID = 411909, PID père = 411907
Fils droit : PID = 411910, PID père = 411906
Fils droit : PID = 411911, PID père = 411907
Fils gauche : PID = 411912, PID père = 411908
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ Fils gauche : PID = 411914, PID père = 411910
Fils gauche : PID = 411915, PID père = 411911
Fils gauche : PID = 411913, PID père = 411909
Fils droit : PID = 411916, PID père = 411908
Fils droit : PID = 411917, PID père = 411909
Fils droit : PID = 411918, PID père = 411910
Fils droit : PID = 411919, PID père = 411911

```

FIGURE 11 – creerA 3 sans attente : affichage non ordonné, le shell reprend avant la fin

Avec wait :

```

*hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex3$ ./creerA 3
Racine : PID = 411994
Fils droit : PID = 411996, PID père = 411994
Fils gauche : PID = 411995, PID père = 411994
Fils gauche : PID = 411998, PID père = 411995
Fils droit : PID = 412000, PID père = 411995
Fils gauche : PID = 411997, PID père = 411996
Fils droit : PID = 411999, PID père = 411996
Fils gauche : PID = 412001, PID père = 411998
Fils gauche : PID = 412002, PID père = 412000
Fils droit : PID = 412003, PID père = 411998
Fils gauche : PID = 412004, PID père = 411997
Fils gauche : PID = 412005, PID père = 411999
Fils droit : PID = 412006, PID père = 412000
Fils droit : PID = 412007, PID père = 411997
Fils droit : PID = 412008, PID père = 411999

```

FIGURE 12 – creerA 3 avec attente : tous les processus terminent avant leur père

## Analyse

Pour une hauteur de 3, l'arbre compte  $2^3 - 1 = 7$  nœuds internes et  $2^3 = 8$  feuilles, soit 14 processus fils au total. L'ordre d'affichage est non déterministe car les fils gauche et droit de chaque nœud s'exécutent en parallèle. Avec `wait`, chaque père attend ses deux fils avant de se terminer, garantissant qu'aucun processus zombie n'est créé.

## 4 Exercice 4 : Gestion des entrées/sorties

### 4.1 Principe

L'objectif est d'observer le comportement du partage d'un fichier entre un père et un fils créé par `fork()`. Le programme ouvre un fichier texte, crée un fils, puis les deux processus lisent le fichier caractère par caractère en affichant leur PID avec chaque caractère lu, en dormant 2 secondes entre chaque lecture.

### 4.2 Version 1 — `fopen` / `fgetc` (E/S bufférisées)

Listing 7 – `ex4.c` — Lecture avec `fopen` / `fgetc`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <fcntl.h>
6
7 int main(int argc, char *argv[]) {
8     if (argc != 2) {
9         fprintf(stderr, "Usage: %s fichier\n", argv[0]);
10        exit(1);
11    }
12
13    FILE *f = fopen(argv[1], "r");
14    if (f == NULL) { perror("fopen"); exit(1); }
15
16    pid_t pid = fork();
17    if (pid < 0) { perror("fork"); exit(1); }
18
19    int c;
20    while ((c = fgetc(f)) != EOF) {
21        printf("PID = %d : '%c'\n", getpid(), c);
22        sleep(2);
23    }
24
25    fclose(f);
26    if (pid > 0) wait(NULL);
27
28    return 0;
29 }
```

## Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex4$ ./ex4 test.txt
PID = 412981 : 'S'
PID = 412981 : 't'
PID = 412981 : 'e'
PID = 412981 : 'p'
PID = 412981 : 'h'
PID = 412981 : 'e'
PID = 412981 : 'n'
PID = 412981 : ' '
PID = 412981 : 'H'
PID = 412981 : 'a'
PID = 412981 : 'w'
PID = 412981 : 'k'
PID = 412981 : 'i'
PID = 412981 : 'n'
PID = 412981 : 'g'
PID = 412981 : '
'
```

FIGURE 13 – Avec `fopen/fgetc` : un seul PID lit tous les caractères

## Analyse

On observe que **tous les caractères sont lus par le même PID**. Cela s'explique par le mécanisme de **buffering** de `fopen` : lors de l'ouverture du fichier, la bibliothèque C charge le contenu dans un buffer interne en mémoire. Lors du `fork()`, ce buffer est **copié** dans l'espace mémoire du fils. Les deux processus possèdent donc chacun leur propre copie du buffer et lisent indépendamment l'intégralité du fichier, sans se partager la position de lecture.

## 4.3 Version 2 — `open / read` (E/S non bufférisées)

### Code source

Listing 8 – `ex4_v2.c` — Lecture avec `open / read`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <fcntl.h>
6
7 int main(int argc, char *argv[]) {
8     if (argc != 2) {
9         fprintf(stderr, "Usage: %s fichier\n", argv[0]);
10        exit(1);
11    }
12
13    int fd = open(argv[1], O_RDONLY);
14    if (fd < 0) { perror("open"); exit(1); }
```



```

15
16     pid_t pid = fork();
17     if (pid < 0) { perror("fork"); exit(1); }
18
19     char c;
20     int n;
21     while ((n = read(fd, &c, 1)) > 0) {
22         printf("PID = %d : '%c'\n", getpid(), c);
23         sleep(2);
24     }
25
26     close(fd);
27     if (pid > 0) wait(NULL);
28
29     return 0;
30 }

```

## Résultat d'exécution

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex4$ ./ex4_v2 test.txt
PID = 413597 : 'S'
PID = 413598 : 't'
PID = 413597 : 'e'
PID = 413598 : 'p'
PID = 413597 : 'h'
PID = 413598 : 'e'
PID = 413597 : 'n'
PID = 413598 : ' '
PID = 413597 : 'H'
PID = 413598 : 'a'
PID = 413597 : 'w'
PID = 413598 : 'k'
PID = 413597 : 'i'
PID = 413598 : 'n'
PID = 413597 : 'g'
PID = 413598 : ' '

```

FIGURE 14 – Avec open/read : les deux PID alternent sur les caractères

## Analyse

Cette fois, les deux PID **alternent** sur les caractères : le père lit 'S', le fils lit 't', le père lit 'e', etc. Avec `open()`, le descripteur de fichier `fd` est une référence vers une **entrée de la table des fichiers ouverts du noyau**, qui contient la position de lecture courante. Lors du `fork()`, le fils hérite d'une **copie du descripteur** pointant vers la **même entrée noyau**. Chaque `read()` avance cette position partagée, ce qui fait que les deux processus se répartissent les caractères sans doublon.

## 4.4 Conclusion

|              | fopen/fgetc     | open/read           |
|--------------|-----------------|---------------------|
| Type d'E/S   | Bufférisée      | Non bufférisée      |
| Après fork() | Copie du buffer | Descripteur partagé |
| Comportement | Chacun lit tout | Ils se partagent    |

## 5 Exercice 5 : La commande execvp

### 5.1 Principe

Le programme `monexec` permet de lancer n'importe quelle commande passée en argument, exactement comme le shell. Il crée un fils avec `fork()`, puis le fils se remplace par la commande demandée via `execvp()`. Le père attend la fin du fils avec `wait()`.

L'astuce clé est l'utilisation de `&argv[1]` : on décale le tableau d'arguments d'un cran pour passer directement la commande et ses arguments à `execvp()`, sans avoir à les recopier.

Listing 9 – `monexec.c` — Lancement d'une commande en argument

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc < 2) {
8         fprintf(stderr, "Usage: %s commande [arguments]\n", argv
9             [0]);
10        exit(1);
11    }
12
13    pid_t pid = fork();
14
15    if (pid < 0) {
16        perror("fork");
17        exit(1);
18    }
19
20    if (pid == 0) {
21        execvp(argv[1], &argv[1]);
22        perror("execvp");
23        exit(1);
24    }
25
26    wait(NULL);
27
28    return 0;
29 }
```

## 5.2 Résultat d'exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex5$ ./monexec touch test.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex5$ ./monexec ls
Makefile monexec monexec.c test.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP4/ex5$
```

FIGURE 15 – `monexec touch test.txt` puis `monexec ls` : les deux commandes s'exécutent correctement

## 5.3 Analyse

Le tableau `argv` du programme est utilisé directement :

| Index                | Valeur                       |
|----------------------|------------------------------|
| <code>argv[0]</code> | "monexec"                    |
| <code>argv[1]</code> | "touch" ← nom de la commande |
| <code>argv[2]</code> | "test.txt" ← argument        |

L'appel `execvp(argv[1], &argv[1])` passe "touch" comme commande et ["touch", "test.", NULL] comme tableau d'arguments. Si `execvp` réussit, le fils est remplacé et ne revient jamais. Le `perror` qui suit n'est atteint qu'en cas d'erreur (commande introuvable par exemple).

## Conclusion

Ce TP4 a permis d'explorer les mécanismes fondamentaux de la programmation système autour des processus. À travers les exercices, nous avons manipulé `fork()` pour créer des processus fils, `wait()` pour synchroniser leur terminaison, et `execvp()` pour remplacer un processus par une commande système.

L'exercice sur les E/S a mis en évidence une distinction importante entre les E/S bufférisées (`fopen/fgetc`) et non bufférisées (`open/read`) : seules ces dernières permettent un vrai partage du descripteur de fichier entre père et fils après un `fork()`.

Enfin, les exercices de synchronisation ont illustré l'importance du placement des `wait()` pour contrôler l'ordre d'exécution et éviter les processus zombies, que ce soit dans une structure en largeur, en profondeur ou en arbre binaire.